

Milvus使用指南

讲师：尚硅谷-宋红康

官网：[尚硅谷](https://www.docker.com/)

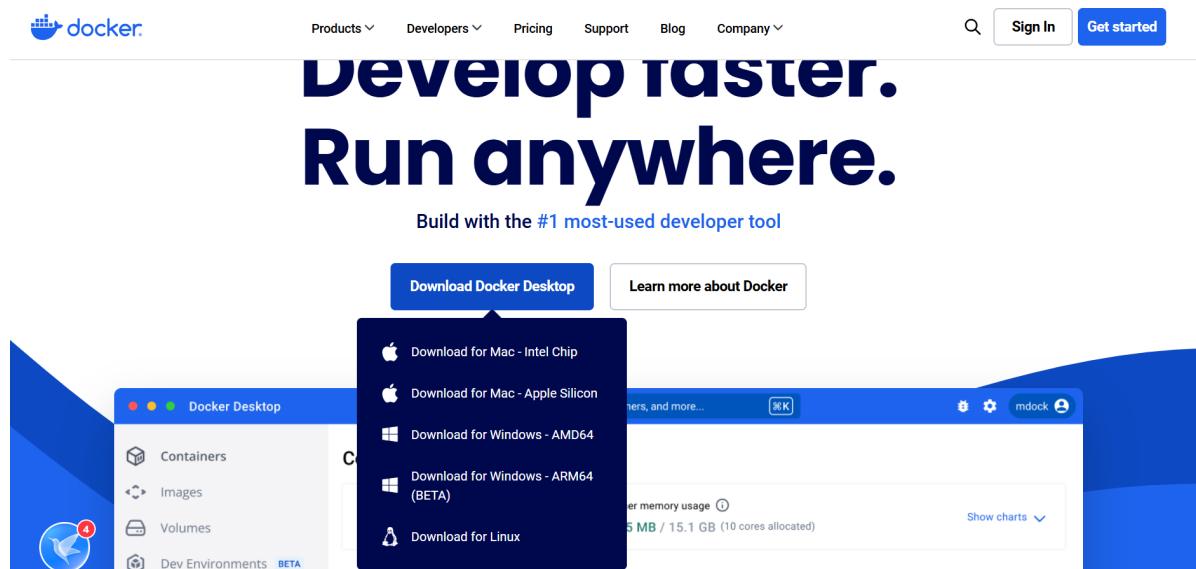
1、安装Docker DeskTop

生产环境推荐Docker安装

1.1 下载安装包

官网：<https://www.docker.com/>

方式1：选择版本下载：



方式2：从视频资料里获取

LangChain V1.2从入门到实战 > 02-资料 > 在 02-资料 中

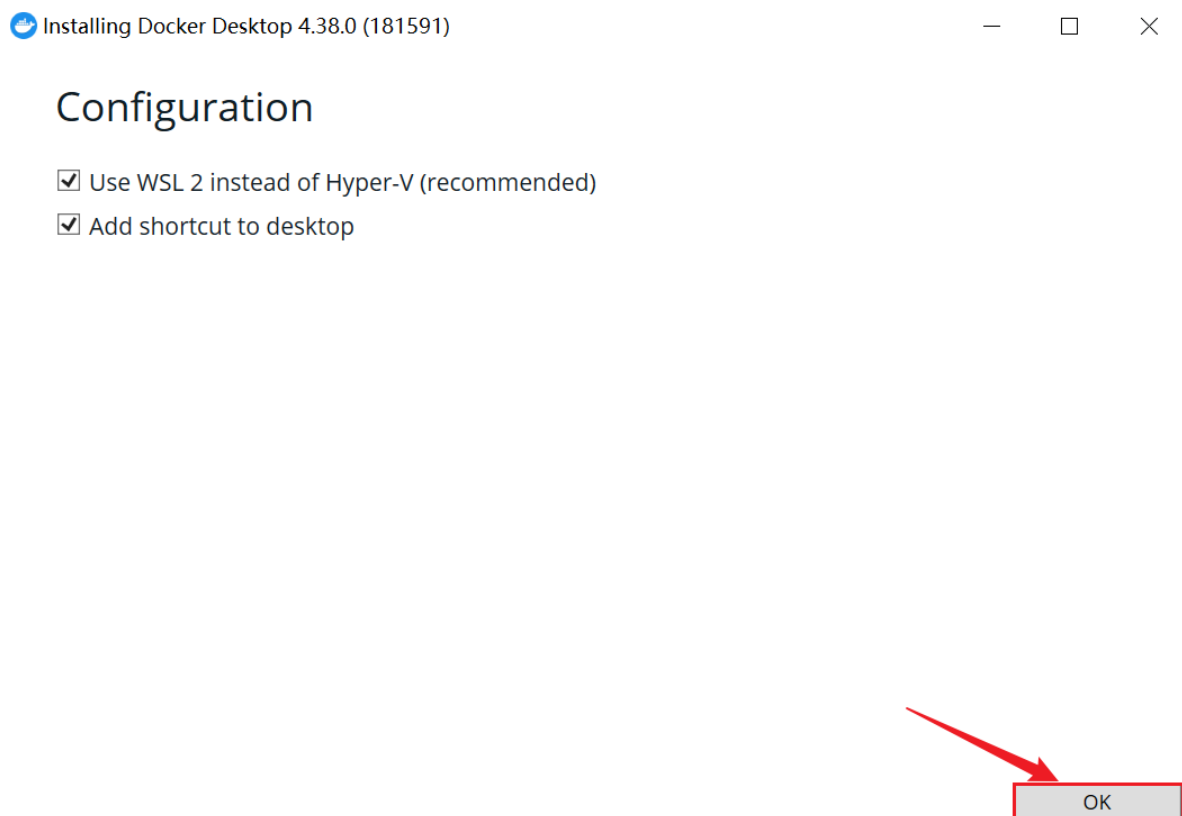
排序

查看

名称	修改日期	类型	大小
asset	06/13 20:49	文件夹	
conda使用指南	06/13 20:49	文件夹	
images	06/14 16:32	文件夹	
Deepseek思考模型工具调用失败解决方案.md	05/12 18:46	MD 文件	
knowledge.txt	04/23 14:46	EditPlus	
Linux云服务器安装与PostgreSQL安装.md	06/12 22:09	MD 文件	
Milvus使用指南.md	06/14 16:34	MD 文件	
Miniconda3-26.1.1-Windows-x86_64.exe	05/13 18:44	应用程序	96
node-v24.14.0-x64.msi	03/13 10:21	Windows Installer 程...	31
requirements_full.txt	06/13 22:30	EditPlus	
如何激活Pycharm 2025.3版本.zip	05/15 10:27	360压缩 ZIP 文件	
Docker Desktop Installer.exe	11/11 16:05	应用程序	574

1.2 安装

步骤1: 点击OK即可



步骤2: 等待安装完成

Docker Desktop 4.38.0

Unpacking files...

```
Unpacking file: resources/docker-desktop.iso
Unpacking file: resources/ddvp.ico
Unpacking file: resources/config-options.json
Unpacking file: resources/componentsVersion.json
Unpacking file: resources/bin/docker-compose
Unpacking file: resources/bin/docker
Unpacking file: resources/.gitignore
Unpacking file: InstallerCli.pdb
Unpacking file: InstallerCli.exe.config
Unpacking file: frontend/vk_swiftshader_icd.json
Unpacking file: frontend/v8_context_snapshot.bin
Unpacking file: frontend/snapshot_blob.bin
Unpacking file: frontend/resources/regedit/vbs/wsRegReadListStream.wsf
Unpacking file: frontend/resources/regedit/vbs/wsRegReadList.wsf
```

步骤3: 重启电脑或稍后自行重启

Docker Desktop 4.38.0

Installation succeeded

You must restart Windows to complete installation.

Close and restart

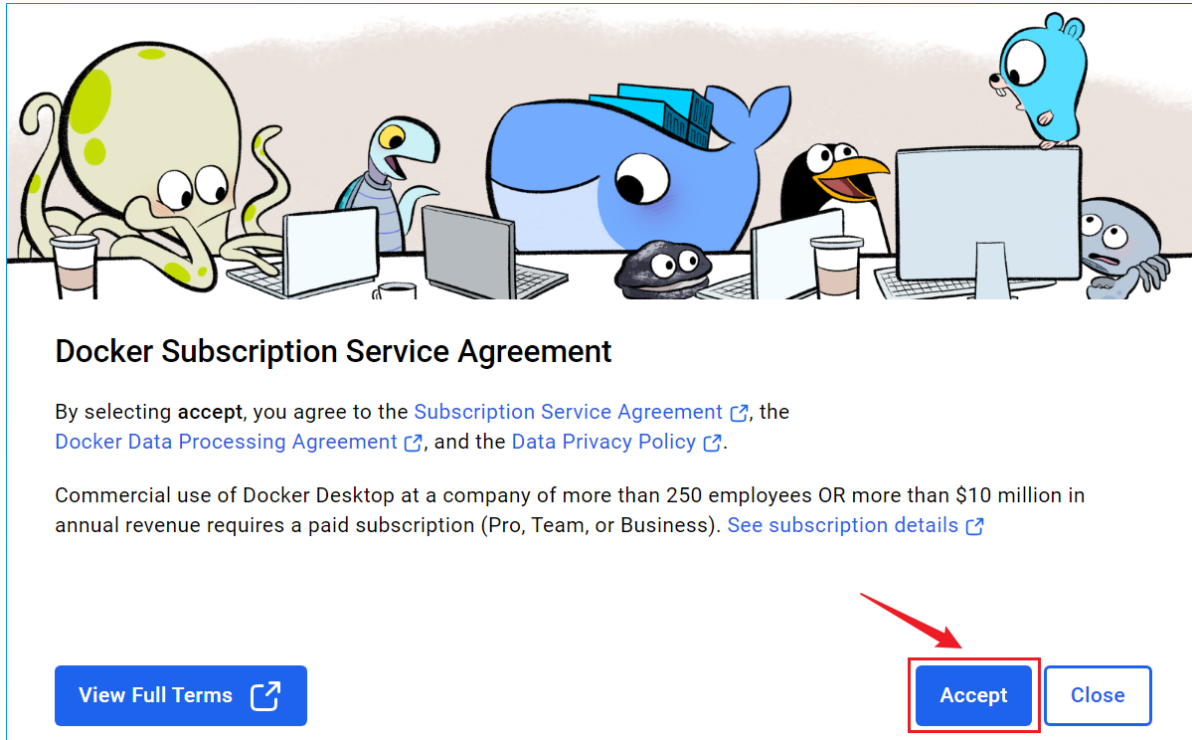
也可以关闭窗口，稍后自行重启。

步骤4: 接受服务协议

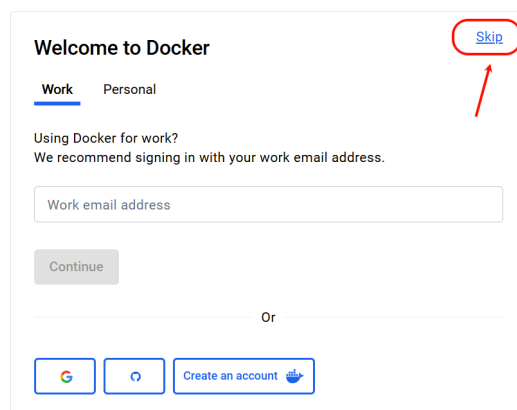
点击启动图标：



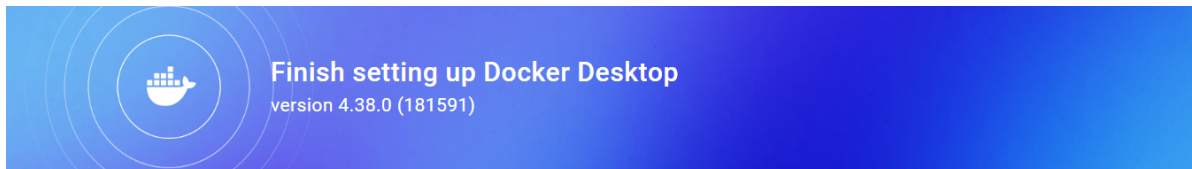
弹窗如下窗口：



选择跳过：



步骤5：完成安装



Complete the installation of Docker Desktop.

☒ Use recommended settings (requires administrator password)

Docker Desktop automatically sets the necessary configurations that work for most developers.

☐ Use advanced settings

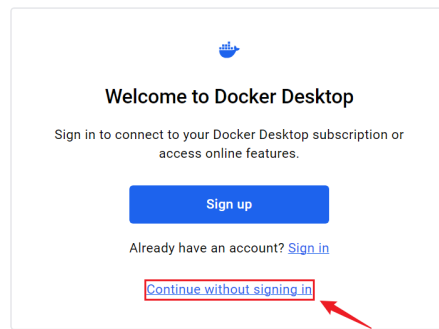
You manually set your preferred configurations.

Finish

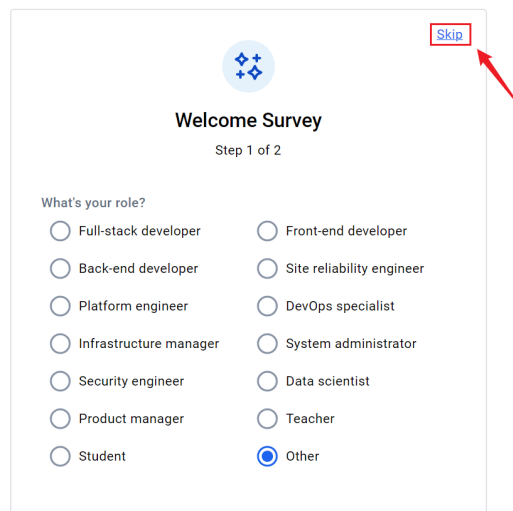
步骤6: 选择允许控制



步骤7: 不登录使用



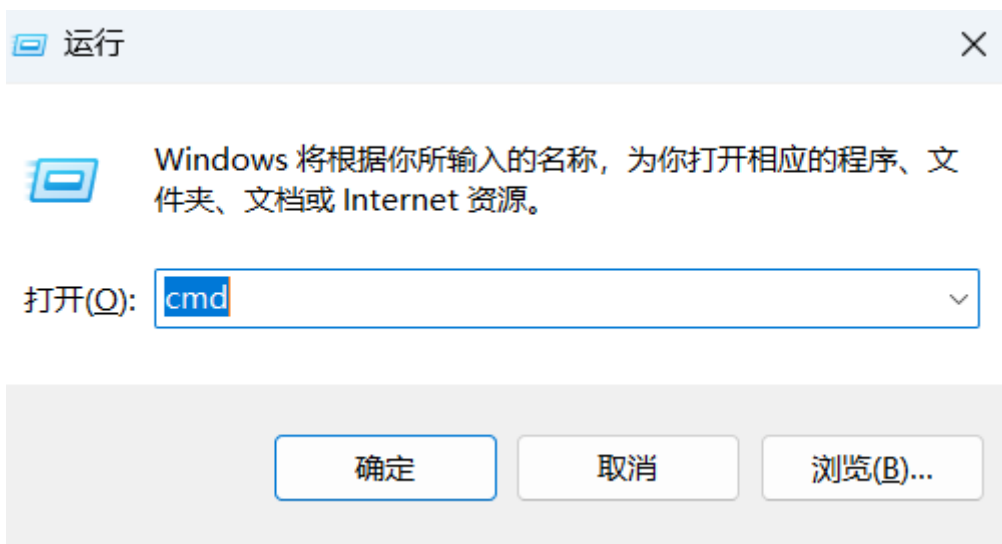
步骤8: 跳过



个别首次安装的小伙伴会被windows系统提示需要安装 **适用于Linux的Windows子系统**。这里选择确认安装。稍等片刻后会完成安装。

步骤9: 安装成功验证

通过win+r调取运行：输入cmd



输入: docker。能显示如下内容即可成功

```
C:\Users\shkst>docker

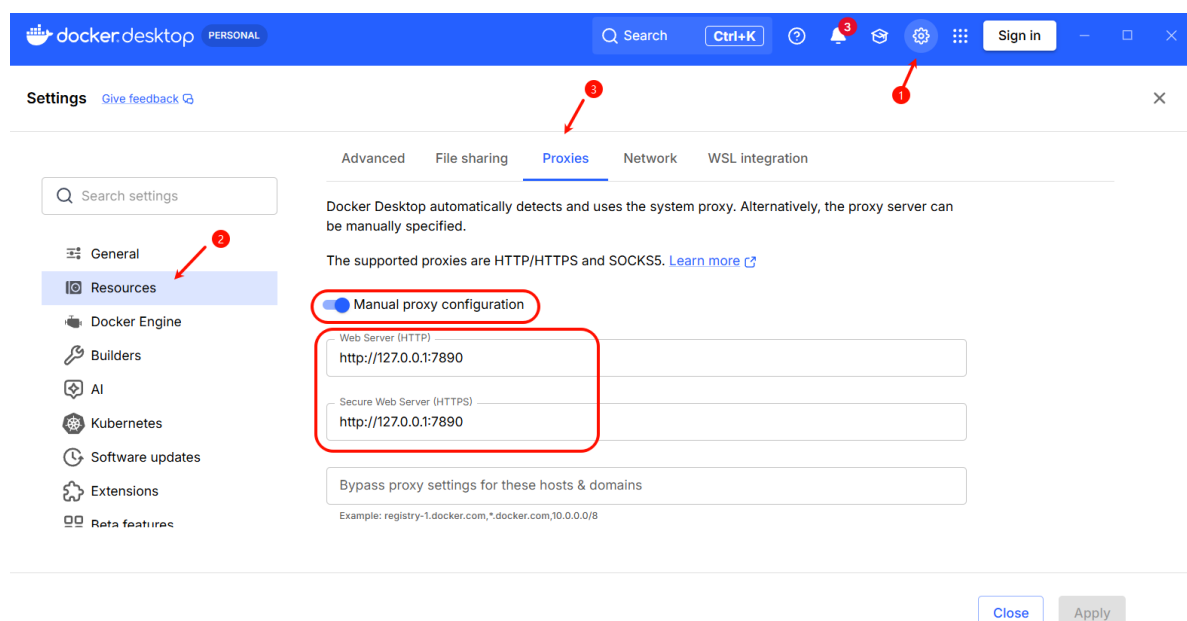
Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run       Create and run a new container from an image
  exec      Execute a command in a running container
  ps        List containers
  build     Build an image from a Dockerfile
  pull      Download an image from a registry
  push      Upload an image to a registry
  images    List images
  login     Authenticate to a registry
  logout    Log out from a registry
  search    Search Docker Hub for images
  version   Show the Docker version information
  info      Display system-wide information

Management Commands:
  ai*       Ask Gordon - Docker Agent
  builder   Manage builds
  buildx*   Docker Buildx
  compose*  Docker Compose
  container Manage containers
  context   Manage contexts
```

1.3 设置Proxy



我们后续下载的Milvus是最新版，在国内可能没有对应镜像，所有建议“科学上网”（开tizi）

1.4 配置加速器(备选)

因为 Milvus 镜像文件比较大（好几百 MB 甚至上 GB），加上 Docker Hub 的官方镜像源在国内访问经常受到网络限制或干扰，所以会出现**下载速度极慢、甚至卡死报错**的情况。所以，我们可以加速器。

- 南京大学镜像站：<https://docker.dockerimages.jobcher.com>
- 阿里云的个人专属加速器（免费且稳定）。
 - 登录 <https://cr.console.aliyun.com/cn-hangzhou/instances/mirrors>，在左侧导航栏选择**镜像工具** -> **镜像加速器**，在页面中你会看到一个属于你的**加速器地址**。

在Docker Desktop中-右上角的 **齿轮图标** -左侧配置 **Docker Engine**：

```
1 {
2   "builder": {
3     "gc": {
4       "defaultKeepStorage": "20GB",
5       "enabled": true
6     }
7   },
8   "experimental": false,
9   "registry-mirrors": [
10    "https://rlmws9iw.mirror.aliyuncs.com",
11    "https://docker.dockerimages.jobcher.com"
12  ]
13 }
```

2、安装Milvus

1. 以管理员身份打开Docker Desktop，确保Docker Engine状态正常后进行下一步
2. 打开powershell，后续操作都在powershell命令行完成
3. 切换文件目录

```
1 (base) PS C:\WINDOWS\system32> d:
2
3 (base) PS D:\> cd D:\developTools\Milvus
4
5 (base) PS D:\developTools\Milvus>
```

注意：developTools\Milvus是你期望保存Milvus数据库的目录，执行前需要确保此文件目录存在。不存在，提前创建好。

4. 下载启动脚本

```
1 (base) PS D:\developTools\Milvus> Invoke-WebRequest
https://raw.githubusercontent.com/milvus-
io/milvus/refs/heads/master/scripts/standalone_embed.bat -OutFile
standalone.bat
```



```
管理员: Windows PowerShell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！https://aka.ms/PSWindows

加载个人及系统配置文件用了 1905 毫秒。
(base) PS C:\WINDOWS\system32> d:
(base) PS D:\> cd D:\developTools\Milvus
(base) PS D:\developTools\Milvus> Invoke-WebRequest https://raw.githubusercontent.com/milvus-io/milvus/refs/heads/master/scripts/standalone_embed.bat -OutFile standalone.bat
(base) PS D:\developTools\Milvus>
```

说明：

- 1、执行完以后，会在当前路径下生成standalone.bat文件。
- 2、如果下载不畅，可以打开tizi，如果没有tizi，可以从02资料里获取此文件。

5. 执行命令

```
1 | (base) PS D:\developTools\Milvus> .\standalone.bat start
```

第三步我们将启动脚本安装在当前目录下，通过bat文件传递启动命令。

```
(base) PS D:\developTools\Milvus> .\standalone.bat start
Unable to find image 'milvusdb/milvus:v3.0-beta' locally
v3.0-beta: Pulling from milvusdb/milvus
4f4fb700ef54: Download complete
6a6ca607f008: Download complete
4974e612dbc8: Downloading [>] 1.049MB/403.1MB
9840082adac6: Download complete
7646c8da3324: Downloading [=====] 4.194MB/29.53MB
9e35a16b5009: Downloading [>] 2.097MB/479.2MB
f5e588a8a82d: Downloading [==>] 4.194MB/75.99MB
01bde5e3c399: Downloading [=>] 1.049MB/33.19MB
1118f1c04d43: Download complete
c0d5a16599a3: Download complete
```

如果更换了启动脚本的下载位置，更改命令指向该文件即可。

6. 查看状态

```
(base) PS D:\developTools\Milvus> .\standalone.bat start
Unable to find image 'milvusdb/milvus:v3.0-beta' locally
v3.0-beta: Pulling from milvusdb/milvus
7646c8da3324: Pull complete
4974e612dbc8: Pull complete
9e35a16b5009: Pull complete
9840082adac6: Pull complete
4f4fb700ef54: Pull complete
f5e588a8a82d: Pull complete
c0d5a16599a3: Pull complete
01bde5e3c399: Pull complete
6a6ca607f008: Pull complete
1118f1c04d43: Pull complete
Digest: sha256:0a030f267a16358901c662adac02813027ace9e23bfbe6db66528e4fd1c28168
Status: Downloaded newer image for milvusdb/milvus:v3.0-beta
Wait for Milvus Starting...
Start successfully.
To change the default Milvus configuration, edit user.yaml and restart the service.
(base) PS D:\developTools\Milvus>
```

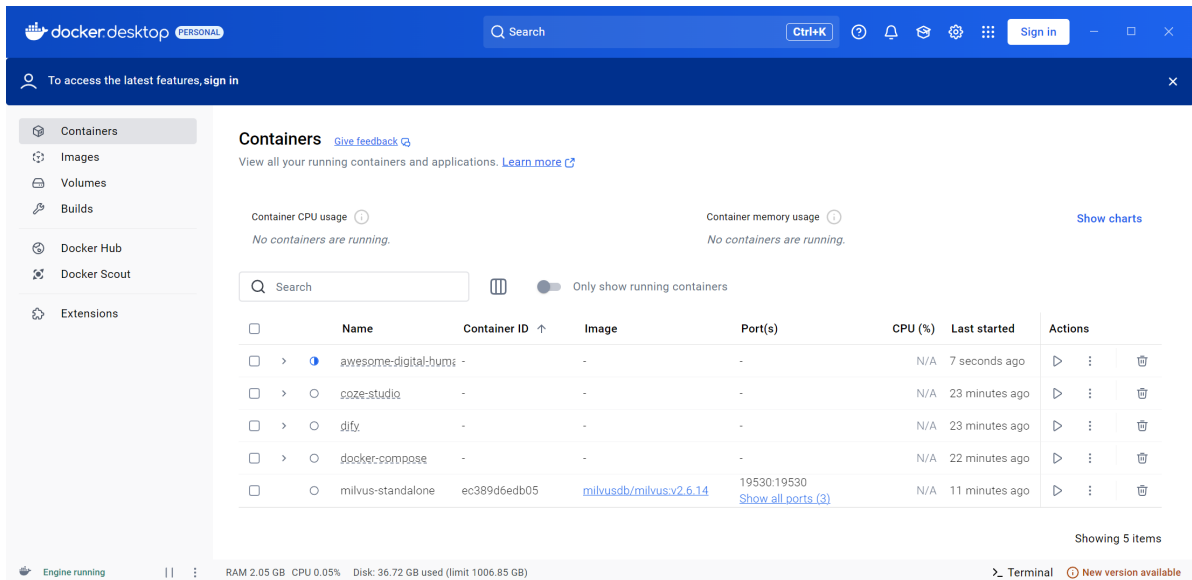
本地缺少Milvus镜像会自动从云端拉取，看到上图所示 **Start successfully** 则启动成功。

7. 停止容器

```
1 | (base) PS D:\developTools\Milvus> .\standalone.bat stop
```

```
1118f1c04d43: Pull complete
Digest: sha256:0a030f267a16358901c662adac02813027ace9e23bfbe6db66528e4fd1c28168
Status: Downloaded newer image for milvusdb/milvus:v3.0-beta
Wait for Milvus Starting...
Start successfully.
To change the default Milvus configuration, edit user.yaml and restart the service.
(base) PS D:\developTools\Milvus> .\standalone.bat stop
Stop successfully.
(base) PS D:\developTools\Milvus>
```

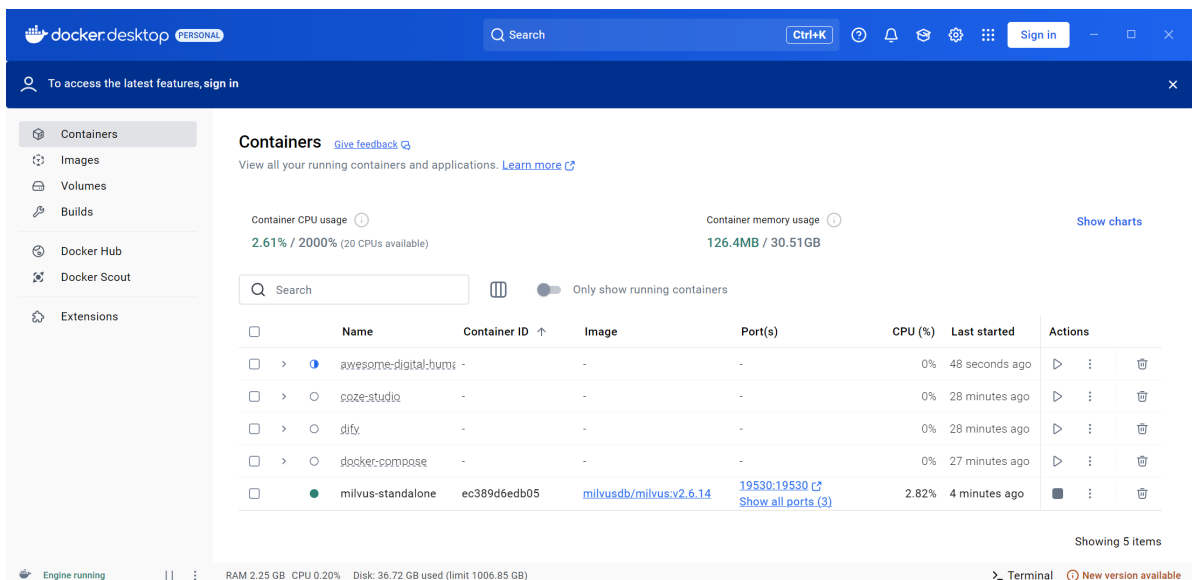
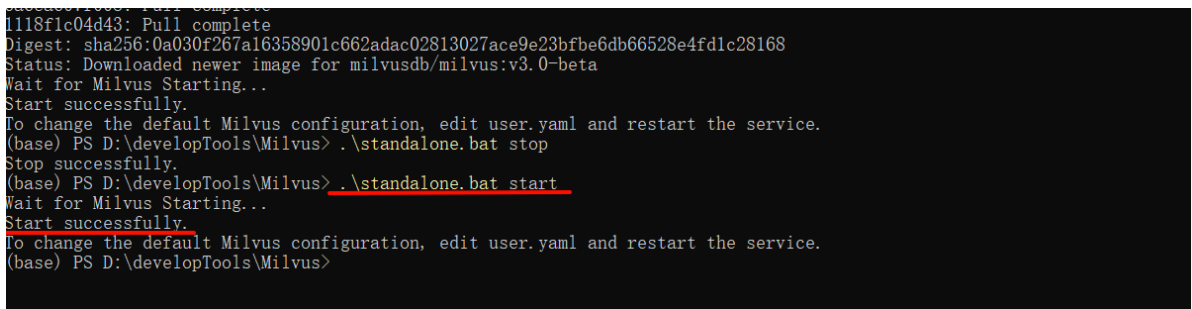
此外，在Docker容器中可见：



3、测试前准备工作

3.1 确保Milvus已启动

```
1 (base) PS D:\developTools\Milvus> .\standalone.bat start
```



3.2 安装Milvus Python SDK

```
1 pip install pymilvus # 已在requirements_full.txt文件中安装
```

Milvus Lite已包含在Milvus的Python SDK中，此处不用Milvus Lite，直接访问在docker desktop安装的、Standalone模式的Milvus服务。

3.3 Milvus数据模型

3.3.1 整体结构

从业务角度，Milvus数据模型层级如下

```
1 Database -> Collection -> Partition -> Entity
```

Database：Milvus 的数据库，用来隔离不同业务数据。

Collection：最核心的逻辑容器，类似于关系型数据库里的 table。

Partition：分区，是 collection 的子集，不是必须手动创建；一个 collection 至少会有默认 partition。

Entity：可以理解为 collection 中的一条记录。类似于关系型数据库的一行数据。

3.3.2 具体说明

在传统数据库中，如果你想存储用户信息，你会建一张表叫 `user_table`；想存储商品，会建一张表叫 `product_table`。

在向量数据库中，我们要存储的是成千上万个“特征向量”以及它们对应的文本。为了把不同用途的向量分门别类地存放，我们就要创建不同的 **Collection**。比如，`COLLECTION_NAME = "docs"` 就是给这张“表”取名叫 `docs`，专门用来存放你切分好的文档片段向量。

4、基本用法

4.1 DDL操作

4.1.1 Database

① 查看数据库

1. 创建客户端

```
1 from pymilvus import MilvusClient
2
3 client = MilvusClient("http://localhost:19530")
```

2. 列出所有数据库

```
1 existing_dbs = client.list_databases()
2
3 print("== Databases ==")
4 for db in existing_dbs:
5     print(db)
```

输出如下

```
1 == Databases ==
2 default
```

② 创建数据库

```
1 db_name = "rag_tutorial"
2
3 if db_name not in existing_dbs:
4     client.create_database(db_name=db_name)
```

③ 删除数据库

```
1 client.drop_database(db_name=db_name)
```

如果数据库下有Collection则无法删除，需要先删除它的所有Collection才能删除Database

4.1.2 Collection

① 切换数据库

```
1 client.use_database(db_name=db_name)
```

上述命令的作用等价于关系型数据库中的 `use <database>`

② 查看数据库下的collections

```
1 collections = client.list_collections()
2
3 print("== Collections ==")
4 for collection in collections:
5     print(collection)
```

输出如下

```
1 == Collections ==
```

③ 创建collection

```
1 collection_name = "docs"
2
3 client.create_collection(
4     collection_name=collection_name,
5     dimension=1024,
6     metric_type="COSINE"
7 )
```

`dimension`：嵌入向量维度，应和嵌入模型的向量维度保持一致，见下文。

`metric_type` 表示向量相似度的计算方式，`COSINE` 表示余弦相似度。

- 当用户提问时，系统会把提问也变成向量，然后去数据库里找“最相似”的本地文本向量。但怎么定义“相似”呢？向量数据库需要知道计算规则。`COSINE`（余弦相似度）关注的是两个向量在方向上的夹角：
 - 如果两个向量方向完全一致（代表文本意思极度接近），余弦值接近 `1`。
 - 如果方向正交（毫无关系），值接近 `0`。
 - 结论：**在 RAG 检索时，Milvus 会帮你计算用户问题与数据库中所有文本的余弦相似度，并把 **得分 (Score) 从大到小排序**，把得分最高（最相似）的前 K 个片段还给你。（注：除了

COSINE 之外，常见的还有 L2 欧氏距离、IP 内积等。)

④ 删除collection

```
1 if client.has_collection(collection_name=collection_name):
2     client.drop_collection(collection_name=collection_name)
```

4.2 DML操作

确保已删除名为 docs 的collection，下文会重新创建

4.2.1 准备嵌入模型

选择 bge-m3 作为嵌入模型，可以通过硅基流动调用，该模型可以免费调用，如果追求更低的延迟、更稳定的服务，也可以充值后选择带有 pro 前缀的模型，如下所示



这两款产品的模型是一样的，区别只在于资费和服务保障。

注册后获取硅基流动的API_KEY，写入.env文件

```
1 SILICONFLOW_BASE_URL=https://api.siliconflow.cn/v1
2 SILICONFLOW_API_KEY=<YOUR_API_KEY>
```

准备嵌入模型，代码如下

```
1 from langchain.embeddings import init_embeddings
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv(override=True)
6
7 # 初始化嵌入模型
8 embed_model = init_embeddings(
9     model="openai:Pro/BAAI/bge-m3",
10     api_key=os.getenv("SILICONFLOW_API_KEY"),
11     base_url=os.getenv("SILICONFLOW_BASE_URL"),
12 )
```

或者：

```
1 from langchain_openai import OpenAIEmbeddings
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv(override=True)
6 # 初始化嵌入模型
7 embed_model = OpenAIEmbeddings(
8     model="Pro/BAAI/bge-m3", # 免费模型 ID: BAAI/bge-m3
9     base_url=os.getenv("SILICONFLOW_BASE_URL"),
10     api_key=os.getenv("SILICONFLOW_API_KEY"),
11 )
```

4.2.2 准备collection

① 创建collection

```
1 client.create_collection(
2     collection_name=collection_name,
3     dimension=1024,
4     metric_type="COSINE"
5 )
```

根据模型维度，创建匹配的collection。

② 查看collection元数据

```
1 from rich import print as rprint
2
3 # 查看 collection 元数据
4 metadata = client.describe_collection(
5     collection_name=collection_name,
6 )
7
8 rprint(metadata)
```

输出如下

```
1 {
2   'collection_name': 'docs',
3   'auto_id': False,
4   'num_shards': 1,
5   'description': '',
6   'fields': [
7     {
8       'field_id': 100,
9       'name': 'id',
10      'description': '',
11      'type': <DataType.INT64: 5>,
12      'params': {},
13      'is_primary': True
14    },
15    {
16      'field_id': 101,
17      'name': 'vector',
```

```

18         'description': '',
19         'type': <DataType.FLOAT_VECTOR: 101>,
20         'params': {'dim': 1024}
21     }
22 ],
23 'functions': [],
24 'aliases': [],
25 'collection_id': 467030880814894392,
26 'consistency_level': 2,
27 'properties': {'timezone': 'UTC'},
28 'num_partitions': 1,
29 'enable_dynamic_field': True,
30 'enable_namespace': False,
31 'created_timestamp': 467030999785013266,
32 'update_timestamp': 467030999785013266
33 }

```

我们在创建collection时没有指定schema，后者可以理解为表结构，此时Milvus会将collection定义为默认结构。字段信息如下

- **id**：数据ID，作为主键唯一标识数据
- **vector**：数据的嵌入向量

此外，**enable_dynamic_field** 为 **True**，这表示 collection 支持**动态字段**。也就是说，除了预定义的 **id** 和 **vector** 字段之外，在插入数据时还可以携带其他未提前声明的字段，这些字段会被自动写入并 **统一存储在动态字段** 中。这样做的好处是能够在不修改 schema 的情况下，灵活保存额外的业务属性，例如文本内容、标签、时间戳或来源信息等，适合字段结构不固定的场景。

4.2.3 准备数据

① 准备原始数据

```

1 # 准备测试数据
2 texts = [
3     "LangChain 是一个用于构建 LLM 应用的开发框架。",
4     "Milvus 是一个适合 AI 应用的向量数据库。",
5     "RAG 的核心是先检索相关知识，再让大模型生成答案。",
6     "Docker Desktop 可以方便地在本地运行 Milvus Standalone。"
7 ]

```

② 生成嵌入向量

```

1 vectors = embed_model.embed_documents(texts)

```

③ 查看生成的嵌入向量

```

1 print(len(vectors))
2 print(len(vectors[0]))
3 print(vectors[0][:5])

```

输出如下

```

1 4
2 1024
3 [-0.027955636382102966, 0.052097633481025696, 0.042480628937482834,
  0.0016073246952146292, 0.030923642218112946]

```

④ 封装为可以插入的数据格式

由上可知，collection需要 **id字段** 作为主键唯一标识数据，需要 **vector字段** 存储嵌入向量，其余字段可以按需添加。

```
1 data = [  
2     {"id": i, "vector": vectors[i], "text": texts[i], "source": "demo"}  
3     for i in range(len(texts))  
4 ]
```

4.2.4 写入数据

① 插入数据

```
1 # 插入数据  
2 insert_res = client.upsert(  
3     collection_name=collection_name,  
4     data=data  
5 )  
6 print("insert result:", insert_res)
```

upsert可以保证幂等写入，即主键相同时覆盖

输出如下

```
1 | insert result: {'upsert_count': 4, 'ids': [0, 1, 2, 3]}
```

② 手动flush

```
1 # 手动 flush  
2 client.flush(collection_name=collection_name)
```

Milvus不会第一时间将数据落盘，要看到写入效果，我们手动flush，将数据刷写到磁盘

③ 查看collection统计信息

```
1 # 查看 collection 统计  
2 stats = client.get_collection_stats(collection_name=collection_name)  
3 print("collection stats:", stats)
```

输出如下

```
1 | collection stats: {'row_count': 4}
```

4.3 DQL操作

4.3.1 扫描数据

通过 **query_iterator** 扫描collection下的所有数据

```
1 it = client.query_iterator(  
2     collection_name=collection_name,  
3     batch_size=100,  
4     filter="", # 不加过滤 = 扫全部  
5     output_fields=["*"] # 你的显式 schema 字段
```



```

6 )
7
8 i = 0
9 while True:
10     rows = it.next()
11     if not rows:
12         break
13     for row in rows:
14         print("=" * 30, f'-> 第{i + 1}条 <-', "=" * 30)
15         print(row)
16         print("=" * 30, f'-> 第{i + 1}条 <-', "=" * 30)
17         i += 1
18
19 it.close()

```

输出如下

```

1  ===== -> 第1条 <-
   =====
2  {'id': 0, 'vector': [0.06801506131887436, 0.06247688829898834, .....],
   'text': 'LangChain 是一个用于构建 LLM 应用的开发框架。', 'source': 'demo'}
3  ===== -> 第1条 <-
   =====
4  ===== -> 第2条 <-
   =====
5  {'id': 1, 'vector': [0.03384151682257652, 0.057637207210063934,
   .....], 'text': 'Milvus 是一个适合 AI 应用的向量数据库。', 'source':
   'demo'}
6  ===== -> 第2条 <-
   =====
7  ===== -> 第3条 <-
   =====
8  {'id': 2, 'vector': [-0.009978039190173149, 0.09607630968093872,
   .....], 'text': 'RAG 的核心是先检索相关知识，再让大模型生成答案。', 'source':
   'demo'}
9  ===== -> 第3条 <-
   =====
10 ===== -> 第4条 <-
    =====
11 {'id': 3, 'vector': [0.027756735682487488, 0.11396646499633789,
    .....], 'text': 'Docker Desktop 可以方便地在本地运行 Milvus Standalone。',
    'source': 'demo'}
12 ===== -> 第4条 <-
    =====

```

4.3.2 通过主键查询数据

```

1  # 查询
2  res = client.get(
3      collection_name=collection_name,
4      ids=[0, 1, 2, 3],
5  )
6
7  print(len(res))
8
9  for i in range(len(res)):
10     print("=" * 30, f'-> 第{i + 1}条 <-', "=" * 30)
11     print(res[i])
12     print("=" * 30, f'-> 第{i + 1}条 <-', "=" * 30)

```

输出如下

```

1  4
2  ===== -> 第1条 <-
3  =====
4  {'id': 0, 'vector': [0.06801506131887436, 0.06247688829898834,
5  0.1035236120223999, -0.030978884547948837, 0.004772709216922522,
6  .....], 'text': 'LangChain 是一个用于构建 LLM 应用的开发框架。', 'source':
7  'demo'}
8  ===== -> 第1条 <-
9  =====
10 ===== -> 第2条 <-
11 =====
12 {'id': 1, 'vector': [0.03384151682257652, 0.057637207210063934,
13 0.07520284503698349, 0.0011678006267175078, 0.013201077468693256,
14 .....], 'text': 'Milvus 是一个适合 AI 应用的向量数据库。', 'source':
15 'demo'}
16 ===== -> 第2条 <-
17 =====
18 ===== -> 第3条 <-
19 =====
20 {'id': 2, 'vector': [-0.009978039190173149, 0.09607630968093872,
21 0.029030004516243935, 0.005278652533888817, 0.022045450285077095,
22 .....], 'text': 'RAG 的核心是先检索相关知识，再让大模型生成答案。', 'source':
23 'demo'}
24 ===== -> 第3条 <-
25 =====
26 ===== -> 第4条 <-
27 =====
28 {'id': 3, 'vector': [0.027756735682487488, 0.11396646499633789,
29 0.04289725795388222, -0.043288204818964005, -0.029911866411566734,
30 .....], 'text': 'Docker Desktop 可以方便地在本地运行 Milvus Standalone。',
31 'source': 'demo'}
32 ===== -> 第4条 <-
33 =====

```

4.3.3 相似度检索

① 准备查询嵌入

```
1 # 相似度检索
2 query = "什么是向量数据库?"
3 query_vector = embed_model.embed_query(query)
```

② 检索

```
1 results = client.search(
2     collection_name=collection_name,
3     data=[query_vector], # Milvus search 这里仍然要传二维列表
4     limit=3,
5     output_fields=["text", "source"]
6 )
7
8 print("=== search results ===")
9 for hit in results[0]:
10     print(hit)
```

`limit` 表示检索结果最多保留几条数据

`output_fields` 表示输出的实体中展示哪些字段。

结果如下

```
1 === search results ===
2 {'id': 0, 'distance': 0.5897263884544373, 'entity': {'text': 'LangChain
  是一个用于构建 LLM 应用的开发框架。', 'source': 'demo'}}
3 {'id': 1, 'distance': 0.5572514533996582, 'entity': {'text': 'Milvus 是一
  个适合 AI 应用的向量数据库。', 'source': 'demo'}}
4 {'id': 2, 'distance': 0.50614333152771, 'entity': {'text': 'RAG 的核心是先
  检索相关知识，再让大模型生成答案。', 'source': 'demo'}}
```